

旅行AI助手

项目背景: 随着大语言模型的发展，传统的智能问答系统已难以满足复杂场景下的多轮对话和动态任务执行需求。为此，本项目基于 LangChain + LangGraph 框架开发多个Agent + WorkFlow实现了一套智能服务助手，聚焦于多轮对话管理、多任务链路执行和工具调用等能力。

项目描述:

围绕旅游/出行业务场景，结合携程式服务流程，数据涵盖航班信息、酒店信息、景点推荐等核心领域，替代客服和销售人員，通过交互的方式，完成携程旅游系统中各个业务。由AI大模型自主完成：机票预订，改签，酒店查询和订单管理，旅游产品销售和预定等

项目实施:

主要包括业务工具模块、身份信息和权限控制模块、多Agent调度块、状态管理模块。

- 1. 工具函数模块：将携程已有的业务，提取整合成为LangChain工具;依据功能风险将工具划分为“只读工具”与“敏感操作工具”，将所有工具安装业务分类（机票业务，酒店业务，旅游业务，租车业务，公共业务），并与大模型完成交互绑定，使大模型能够动态触发工具调用。
- 2. 身份信息和权限控制模块：主助手智能体（Agent）一开始就能通过节点fetch_user_info从请求参数中获取旅客ID，并调用Tools查询完整的旅客信息，并保存在State中，当整个工作流执行到任何敏感工具节点之前都会进行interrupt_before（中断），并且通过MemorySaver()保存上下文信息，即使中断的情况下Agent也能通过Config中的thread_id和旅客ID来保证上下文统一。
- 3. 多Agent调度块：整个模块主要是把业务流程细分为四个子工作流，由一个主Agent进行居中调度，每个子工作流作为一个单独的Agent负责完成各自的业务，分为：机票业务Agent、酒店业务Agent、旅游业务Agent，租车业务Agent。每个（Agent）子工作流包含五个节点，分别是入口节点(create_entry_node)、决策节点（带有大模型的节点）、只读工具、敏感操作工具和离开当前任务节点（所有子助理共用一个），从主工作流到子工作流的调度是通过动态路由条件(add_conditional_edges)来实现，从子工作流到主工作流是通过控制State中的update_dialog_stack来管理记录
- 4. 状态管理模块:整个模块是State模块，就是保存工作流中的messages（历史消息列表）、用户信息(user_info)、以及工作流状态栈(dialog_stack)。

1创建工具

按不同助手创建所需的工具，有只需要查询数据的安全工具，和需要数据库修改的敏感工具，后续在定义工作流时在调用敏感工具之前需要终端，让用户确认是否要继续执行，安全工具则不用

1.1航班预订助手工具

工具名	类型	作用
<code>search_flights</code>	安全工具（只读）	查询可用航班信息（出发地/目的地/日期）
<code>update_ticket_to_new_flight</code>	敏感工具（写入）	将现有机票改签到新航班
<code>cancel_ticket</code>	敏感工具（写入）	取消已预订的机票

工具名	类型	作用
<code>fetch_user_flight_information</code>	安全工具（只读）	获取用户当前的航班预订信息

1.2 酒店预订助手工具

工具名	类型	作用
<code>search_hotels</code>	安全工具（只读）	搜索符合条件的酒店（位置/日期/价格）
<code>book_hotel</code>	敏感工具（写入）	预订指定酒店
<code>update_hotel</code>	敏感工具（写入）	修改酒店预订信息
<code>cancel_hotel</code>	敏感工具（写入）	取消酒店预订

1.3 租车预订助手工具

工具名	类型	作用
<code>search_car_rentals</code>	安全工具（只读）	查询可租车辆（车型/地点/时间）
<code>book_car_rental</code>	敏感工具（写入）	预订租车服务
<code>update_car_rental</code>	敏感工具（写入）	修改租车订单
<code>cancel_car_rental</code>	敏感工具（写入）	取消租车预订

1.4 旅游活动助手工具

工具名	类型	作用
<code>search_trip_recommendations</code>	安全工具（只读）	查找旅游景点/活动推荐
<code>book_excursion</code>	敏感工具（写入）	预订旅游活动（如一日游）
<code>update_excursion</code>	敏感工具（写入）	修改旅游活动预订
<code>cancel_excursion</code>	敏感工具（写入）	取消旅游活动

1.5 主助理工具

工具名	类型	作用
<code>tavily_tool</code> (TavilySearchResults)	安全工具（只读）	全网搜索旅行相关信息
<code>lookup_policy</code>	安全工具（只读）	查询航空公司/酒店政策

工具名	类型	作用
CompleteOrEscalate	控制工具	将对话控制权交还主助理

对 CompleteOrEscalate 工具的解释：

CompleteOrEscalate 是子agent在识别到需要转换其他agent时调用，用于标记当前任务为已完成和/或将对话的控制权升级到主助理，主助理可以根据用户的需求重新路由对话。

当调用 CompleteOrEscalate 时，LangGraph 会将此调用转换为 messages中的ToolMessage 并存入状态State中

在定义中 CompleteOrEscalate 是一个类，为什么可以当作工具？

- **底层约定：**LangChain 将任何继承 BaseModel 的 Pydantic 类视为 **工具签名** (Tool Signature)
- **自动转换：**框架自动将类定义转换为 OpenAI 标准的工具调用格式

使用类来定义的优势：

- **可复用性：**类可以在其他地方作为数据类型使用
- **自描述性：**通过 Config 类提供更丰富的元数据
- **扩展性：**未来可轻松添加新字段不影响既有逻辑

代码如下

```
class CompleteOrEscalate(BaseModel): # 定义数据模型类
    """
    一个工具，用于标记当前任务为已完成和/或将对话的控制权升级到主助理，
    主助理可以根据用户的需求重新路由对话。
    """

    cancel: bool = True # 默认取消任务
    reason: str # 取消或升级的原因说明

    class Config: # 内部类 Config: json_schema_extra: 这个字段包含了一些示例数据
        json_schema_extra = {
            "example": {
                "cancel": True,
                "reason": "用户改变了对当前任务的想法。",
            },
            "example2": {
                "cancel": True,
                "reason": "我已经完成了任务。",
            },
            "example3": {
                "cancel": False,
                "reason": "我需要搜索用户的电子邮件或日历以获取更多信息。",
            },
        }
```

1.6带错误回退机制的工具节点函数

工具正常调用则返回正常的执行结果

执行失败则触发自己定义的错误处理函数，在 langgraph 中 `exception_key="error"` 会向 state 中动态扩展 error 字段，并将错误信息填入进去

代码如下

```
def handle_tool_error(state) -> dict:
    """
    处理工具调用时发生的错误。

    参数:
        state (dict): 当前的状态字典，包含错误信息和消息列表。

    返回:
        dict: 包含错误信息的新消息列表的字典。
    """
    error = state.get("error") # 获取错误信息
    tool_calls = state["messages"][-1].tool_calls # 获取最后一条消息中的所有工具调用
    return {
        "messages": [
            ToolMessage(
                content=f"错误: {repr(error)}\n请修正您的错误。",
                tool_call_id=tc["id"], # 关联到发生错误的工具调用ID
            )
            for tc in tool_calls # 遍历所有的工具调用并生成对应的消息
        ]
    }

def create_tool_node_with_fallback(tools: list) -> dict:
    """
    创建一个带有回退机制的工具节点。当指定的工具执行失败时（例如抛出异常），将触发回退操作。

    参数:
        tools (list): 工具列表。

    返回:
        dict: 带有回退机制的工具节点，ToolNode工具返回ToolMessage，handle_tool_error函数返回messages。
    """
    return ToolNode(tools).with_fallbacks(
        # 使用handle_tool_error函数作为回退处理
        [RunnableLambda(handle_tool_error)], exception_key="error"
    )
```

1.7打印对话事件的工具函数

定义了一个用于打印对话事件的工具函数 `_print_event`，它在对话系统的工作流执行过程中负责**格式化输出状态变更和消息内容**

代码如下

```
def _print_event(event: dict, _printed: set, max_length=1500):
    """
    打印事件信息，特别是对话状态和消息内容。如果消息内容过长，会进行截断处理以保证输出的可读性。

    参数：
        event (dict): 事件字典，包含对话状态和消息。
        _printed (set): 已打印消息的集合，用于避免重复打印。
        max_length (int): 消息的最大长度，超过此长度将被截断。默认值为1500。
    """
    current_state = event.get("dialog_state")
    if current_state:
        print("当前处于: ", current_state[-1]) # 输出当前的对话状态
    message = event.get("messages")
    if message:
        if isinstance(message, list):
            message = message[-1] # 如果消息是列表，则取最后一个
        if message.id not in _printed:
            msg_repr = message.pretty_repr(html=True)
            if len(msg_repr) > max_length:
                msg_repr = msg_repr[:max_length] + " ... (已截断)" # 超过最大长度
            # 则截断
            print(msg_repr) # 输出消息的表示形式
            _printed.add(message.id) # 将消息ID添加到已打印集合中
```

2定义状态state

状态定义首先是最基本的messages，使用langgraph框架的 `add_messages` 自动添加最新消息

接着根据项目添加特定字段：

`user_info` 记录了旅客的基本航班信息，在工作流的入口节点就会调用相关工具获取旅客信息，并且将信息填充到状态中

`dialog_state` 在多agent中，使用这个字段来记录当前所处agent，在定义时限定了取值范围，分别为5个agent助手名字，使用自定义函数 `update_dialog_stack` 来控制更新逻辑

```
# 状态类
class State(TypedDict):
    """
    定义一个结构化的字典类型，用于存储对话状态信息。
    字段：
        messages (list[AnyMessage]): 使用 Annotated 注解附加了 add_messages 功能的消息列表，
        可能用于自动处理消息的某些方面。
        user_info (str): 存储用户信息的字符串。
        dialog_state (list[Literal["assistant", "update_flight",
        "book_car_rental",
        "book_hotel", "book_excursion"]]): 对话状态栈，
        限定只能包含特定的几个值，
        并使用 update_dialog_stack 函数来控制其更新逻辑。
    """
    messages: Annotated[list[AnyMessage], add_messages]
    user_info: str
    dialog_state: Annotated[
```

```
list[ #其元素严格限定为上述五个字符串值之一。这种做法确保了对话状态管理逻辑的一致性和
      正确性，避免了意外的状态值导致的潜在问题。
      Literal[
          "assistant",
          "update_flight",
          "book_car_rental",
          "book_hotel",
          "book_excursion",
      ]
  ],
  update_dialog_stack,
]
```

控制状态更新的函数如下，在python中list列表也有栈的特性，可以pop出栈与append入栈，在栈中最顶部的是当前调用的agent

```
#Optional让传参可以是None
def update_dialog_stack(left: list[str], right: Optional[str]) -> list[str]:
    """
    更新对话状态栈。
    参数：
        left (list[str]): 当前的状态栈。
        right (Optional[str]): 想要添加到栈中的新状态或动作。如果为 None，则不做任何更改；
                               如果为 "pop"，则弹出栈顶元素；否则将该值添加到栈中。

    返回：
        list[str]: 更新后的状态栈。
    """
    if right is None:
        return left # 如果right是None，保持当前状态栈不变
    if right == "pop":
        return left[:-1] # 如果right是"pop"，移除栈顶元素（即最后一个状态）
    return left + [right] # 否则，将right添加到状态栈中
```

3定义子agent

分别创建每个子agent，以航班预定助手为例

3.1prompt

首先定义助手的prompt，其中除了基本的提示语句外，还使用消息占位符传入：

1. 先前获取到的 `user_info` 供助手获取用户信息，使用标签明确标识 `{user_info}` 的内容属于航班数据，在生成的提示词中形成明显的视觉区块，帮助LLM快速定位关键信息。其他不涉及航班的助手可以不提供 `user_info`
2. 携程相关一系列业务讲究时效性，因此还需传入当前时间 `time`
3. `placeholder`，"`{messages}`" 将之前的对话历史自动注入到当前提示词中，`user_info` 与 `time` 两个没有使用placeholder是因为

变量	类型	变化频率	适合位置
<code>{messages}</code>	动态对话历史	每次调用都不同	<code>placeholder</code>

变量	类型	变化频率	适合位置
<code>{user_info}</code>	静态用户数据	同一会话中基本不变	直接嵌入
<code>{time}</code>	上下文变量	每次调用固定	<code>.partial()</code> 绑定

`placeholder` 是 `ChatPromptTemplate` 中用于声明“此处需要动态填充内容”的标记。它有两个核心特点：

- **占位特性：**在模板构建时不包含实际内容，运行时才被填充
- **变量插值：**通过 `{变量名}` 指定要插入的内容（如 `{messages}`）

3.2Tools

分别创建安全工具（只读操作）和敏感工具（数据库修改操作）的集合

然后将安全工具以及敏感工具合并为一个工具

3.3构建子agent

构建 **可运行对话流程**，将 **提示词模板**、**大语言模型（LLM）** 和 **工具调用能力** 组合成一个可执行的流水线，作为航班预定助手子agent

在绑定工具时需要给每个子agent绑定 `CompleteOrEscalate`，作用：在子agent识别到无法完成需求时调用，切换回主助力，让主agent判断接下来的执行

与传统 Agent 的差异

特性	<code>update_flight_runnable</code> (子代理)	传统独立 Agent
创建方式	通过 Runnable 管道组合	通常继承 Agent 基类
工具范围	严格受限（仅绑定工具）	可能访问全局工具库
状态管理	依赖父图（StateGraph）传递状态	自主管理状态
生命周期	由主代理按需激活/销毁	独立运行

以下是航班预定助手的代码

```
# 航班预订助手
flight_booking_prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "您是专门处理航班查询，改签和预定的助理。"
            "当用户需要帮助更新他们的预订时，主助理会将工作委托给您。"
            "请与客户确认更新后的航班详情，并告知他们任何额外费用。"
            "在搜索时，请坚持不懈。如果第一次搜索没有结果，请扩大查询范围。"
            "如果您需要更多信息或客户改变主意，请将任务升级回主助理。"
            "请记住，在相关工具成功使用后，预订才算完成。"
            "\n\n当前用户的航班信息:\n<Flights>\n{user_info}\n</Flights>"
            "\n当前时间: {time}."
            "\n\n如果用户需要帮助，并且您的工具都不适用，则"
```

```

        "“CompleteOrEscalate”对话给主助理。不要浪费用户的时间。不要编造无效的工具或功能。",
    ),
    ("placeholder", "{messages}"),
]
).partial(time=datetime.now())

# 定义安全工具（只读操作）和敏感工具（涉及更改的操作）
update_flight_safe_tools = [search_flights]
update_flight_sensitive_tools = [update_ticket_to_new_flight, cancel_ticket]

# 合并所有工具
update_flight_tools = update_flight_safe_tools + update_flight_sensitive_tools

# 创建可运行对象，绑定航班预订提示模板和工具集，包括CompleteOrEscalate工具
update_flight_runnable = flight_booking_prompt | llm.bind_tools(
    update_flight_tools + [CompleteOrEscalate]
)

```

4定义子 workflow

这里以构建航班助手的子 workflow 为例

4.1 定义入口节点函数

每一个子 workflow 都需要一个入口节点

为什么需要入口节点？

1. 避免用户看到不连贯的对话（如突然改变回复风格），入口节点会初始化新的对话状态，可以使上下文过渡更加平滑，并且保持任务的连续性
2. 如果直接跳转到子代理可能导致：工具调用历史混淆、对话栈（dialog_state）管理混乱。使用入口节点可以明确标记当前代理类型

入口节点函数为什么要用闭包实现？

保持状态：通过闭包，您可以创建一个携带额外信息（如 assistant_name 和 new_dialog_state）的函数。这些信息在 create_entry_node 函数被调用时确定，并且可以在返回的 entry_node 函数中使用。

封装和复用：闭包使得您可以将特定行为（这里是生成带有特定内容的消息和对话状态）与某些参数（如助理名称和新的对话状态）封装在一起，从而更容易地重用这个逻辑。

延迟执行：闭包允许您定义一个行为或计算，然后在之后某个时刻执行它，这为更灵活的程序设计提供了可能。

具体实现：

create_entry_node 接受两个参数：assistant_name 和 new_dialog_state。它返回一个新的函数 entry_node。

返回的 entry_node 函数接受一个 state 参数，代表当前对话的状态。这就满足了定制节点的要求。然后，构造并返回一个字典，其中包含一条新消息（通过 ToolMessage 类实例化）和更新后的对话状态 (new_dialog_state)。这条消息的内容是根据传入的 assistant_name 定制的，并指导助手如何行动

入口节点代码：

```
def create_entry_node(assistant_name: str, new_dialog_state: str) -> Callable:
```



```
"""
```

这是一个函数工程： 创建一个入口节点函数，当对话状态转换时调用。

该函数生成一条新的对话消息，并更新对话的状态。

```
:param assistant_name: 新助理的名字或描述。
```

```
:param new_dialog_state: 要更新到的新对话状态。
```

```
:return: 返回一个根据给定的assistant_name和new_dialog_state处理对话状态的函数。
```

```
"""
```

```
def entry_node(state: dict) -> dict:
```

```
    """
```

根据当前对话状态生成新的对话消息并更新对话状态。

```
:param state: 当前对话状态，包含所有消息。
```

```
:return: 包含新消息和更新后的对话状态的字典。
```

```
"""
```

```
# 获取最后一个消息中的工具调用ID
```

```
tool_call_id = state["messages"][-1].tool_calls[0]["id"]
```

```
return {
```

```
    "messages": [
```

```
        ToolMessage(
```

```
            content=f"现在助手是{assistant_name}。请回顾上述主助理与用户之间的对话。"
```

```
话。"
```

```
            f"用户的意图尚未满足。使用提供的工具协助用户。记住，您是
```

```
{assistant_name}，"
```

```
            "并且预订、更新或其他操作未完成，直到成功调用了适当的工具。"
```

```
            "如果用户改变主意或需要帮助进行其他任务，请调用
```

```
CompleteOrEscalate函数让主要的主助理接管。"
```

```
            "不要提及你是谁——仅作为助理的代理。",
```

```
            tool_call_id=tool_call_id,
```

```
        )
```

```
    ],
```

```
    "dialog_state": new_dialog_state,
```

```
}
```

```
return entry_node
```

对Callable的解释：

Callable 描述任何支持 `()` 调用操作的对象，包括：

- 普通函数（`def` 定义的函数）
- Lambda 表达式
- 类（调用类会创建实例）
- 实现了 `__call__` 方法的对象

4.2定义退出节点函数

所有子工作流公用一个退出节点

触发条件：当子代理调用 `CompleteOrEscalate` 工具时，路由函数 `route_update_flight` 会返回 `"leave_skill"`，状态机随即跳转到该节点。

为什么需要退出节点？

- 清理子代理上下文：子代理运行时可能产生临时数据（如未完成的表单、工具调用缓存），直接跳转会引发状态污染。
- 维护对话栈：若直接从子代理跳回主代理，会导致 `dialog_state` 栈不一致，后续路由可能崩溃。

为什么退出节点只需要在航班子助手中添加？

- 代码复用设计：所有子代理的退出行为一致（清理状态、返回主代理），无需重复实现，将退出节点定义在一处便于维护。
- LangGraph的工作机制：**节点全局可用**，LangGraph 的状态图允许不同子工作流引用同一节点。一旦通过 `builder.add_node("leave_skill", pop_dialog_state)` 注册节点，所有子助手的路由函数在检测到 `CompleteOrEscalate` 时，都可以返回相同的节点名 `"leave_skill"`。且退出行为由 `dialog_state: "pop"` 触发，与具体哪个子助手调用无关。

退出节点函数代码

```
# 此节点将用于所有子助理的退出
def pop_dialog_state(state: dict) -> dict:
    """
    弹出对话栈并返回主助理。
    这使得完整的图可以明确跟踪对话流，并根据需要委托控制给特定的子图。
    :param state: 当前对话状态字典
    :return: 包含新的对话状态和消息的字典
    """
    messages = []
    if state["messages"][-1].tool_calls:
        # 注意：目前不处理LLM同时执行多个工具调用的情况
        messages.append(
            ToolMessage(
                content="正在恢复与主助理的对话。请回顾之前的对话并根据需要协助用户。",
                tool_call_id=state["messages"][-1].tool_calls[0]["id"],
            )
        )
    return {
        "dialog_state": "pop", # 更新对话状态为弹出
        "messages": messages, # 返回消息列表
    }
```

4.3自定义表示节点的类

自定义一个名为 `CtripAssistant` 的类作为 **统一的可执行节点封装器**

使用一个自定义的类作为节点将不同的 `Runnable` 对象（如主助理和航班助手）包装成 **统一接口的节点处理器**。

使用这个类类作为节点，在进入某一个助手前可以**检查**输入是否靠谱、**重试**不合理的操作等。使整个系统就像一家服务周到的餐厅——即使客人刁难，也能从容应对！

代码如下

```
class CtripAssistant:

    # 自定义一个类，表示流程图的一个节点（复杂的）
```

```

def __init__(self, runnable: Runnable):
    """
    初始化助手的实例。
    :param runnable: 可以运行对象，通常是一个Runnable类型的
    """
    self.runnable = runnable

def __call__(self, state: State, config: RunnableConfig):
    """
    调用节点，执行助手任务
    :param state: 当前工作流的状态
    :param config: 配置：里面有旅客的信息
    :return:
    """
    while True:
        # 创建了一个无限循环，它将一直执行直到：从 self.runnable 获取的结果是有效的。
        # 如果结果无效（例如，没有工具调用且内容为空或内容不符合预期格式），循环将继续执
        # configuration = config.get('configurable', {})
        # user_id = configuration.get('passenger_id', None)
        # state = {**state, 'user_info': user_id} # 从配置中得到旅客的ID，也追加
        # 到state

        result = self.runnable.invoke(state)
        # 如果，runnable执行完后，没有得到一个实际的输出
        if not result.tool_calls and ( # 如果结果中没有工具调用，并且内容为空或内容
            # 列表的第一个元素没有"text"，则需要重新提示用户输入。
            not result.content
            or isinstance(result.content, list)
            and not result.content[0].get("text")
        ):
            messages = state["messages"] + [{"user", "请提供一个真实的输出作为回
            应。"}]

            state = {**state, "messages": messages}
        else: # 如果：runnable执行后已经得到，想要的输出，则退出循环
            break
    return {'messages': result}

```

4.4构建子工作流

以航班助手的子工作流为例

1. 添加入口节点 `enter_update_flight`
2. 添加处理航班更新的实际节点 `update_flight`，使用 `CtripAssistant` 类封装
3. 连接入口节点到实际处理节点
4. 分别添加敏感工具 `update_flight_sensitive_tools` 与安全工具 `update_flight_safe_tools` 的节点，工具节点使用1.6中定义好的工具节点函数封装（保护性代码）
5. 创建敏感工具与安全工具返回 `update_flight` 的边（非条件便直接添加）
6. 添加条件边，由 `update_flight` 到敏感工具节点、安全工具节点、离开 `update_flight` 的节点 `leave_skill` 或终止节点END（每个子工作流都可以终止本轮对话，不需要再返回主助理然后再终止）

7. 定义条件边的路由函数

- 检查是否满足对话终止条件，——END
- 检查是否调用了CompleteOrEscalate，——leave_skill
- 检查**所有**调用的工具是否都是安全工具，——update_flight_safe_tools
- 只剩敏感工具调用，——update_flight_sensitive_tools

路由函数代码如下

```
def route_update_flight(state: dict):  
    """  
    根据当前状态路由航班更新流程。  
  
    :param state: 当前对话状态字典  
    :return: 下一步应跳转到的节点名  
    """  
  
    route = tools_condition(state) # 判断下一步的方向  
    if route == END:  
        return END # 如果结束条件满足，则返回END  
    tool_calls = state["messages"][-1].tool_calls # 获取最后一条消息中的工具调用  
  
    did_cancel = any(tc["name"] == CompleteOrEscalate.__name__ for tc in tool_calls) # 检查是否调用了CompleteOrEscalate  
    if did_cancel:  
        return "leave_skill" # 如果用户请求取消或退出，则跳转至leave_skill节点  
    safe_tool_names = [t.name for t in update_flight_safe_tools] # 获取所有安全工具的名字  
    if all(tc["name"] in safe_tool_names for tc in tool_calls): # 如果所有调用的工具都是安全工具  
        return "update_flight_safe_tools" # 跳转至安全工具处理节点  
    return "update_flight_sensitive_tools" # 否则跳转至敏感工具处理节点
```

函数部分内容的解释：

- tools_condition(state) 是LangGraph 内置函数，检查是否满足对话终止条件（如用户说"结束"或任务完成）
- state["messages"][-1] 获取消息历史中的最新一条消息，tool_calls 提取该消息中所有工具调用（可能包含多个并行调用）
- any(tc["name"] == CompleteOrEscalate.__name__ for tc in tool_calls)

检查工具调用列表 tool_calls 中 **是否存在** 名为 "CompleteOrEscalate" 的调用（即子代理请求退出或升级到主代理）。

逐层解析

部分	说明	示例值
tool_calls	当前消息中的工具调用列表	[{"name": "search_flights"}, {"name": "CompleteOrEscalate"}]
tc["name"]	单个工具调用的名称	"CompleteOrEscalate"

部分	说明	示例值
<code>CompleteOrEscalate.__name__</code>	类名转字符串	<code>"CompleteOrEscalate"</code>
<code>any(...)</code>	如果列表中有任意元素满足条件则返回 <code>True</code>	<code>True</code> （因存在匹配项）

◦ `all(tc["name"] in safe_tool_names for tc in tool_calls)`

验证工具调用列表 `tool_calls` 中 **所有工具** 是否都属于安全工具（即仅包含在 `safe_tool_names` 中定义的只读操作）。

逐层解析

部分	说明	示例值
<code>safe_tool_names</code>	预定义的安全工具名称列表	<code>["search_flights"]</code>
<code>tc["name"] in safe_tool_names</code>	检查单个工具是否安全	<code>True</code> （如果是 <code>search_flights</code> ）
<code>all(...)</code>	仅当所有元素满足条件时返回 <code>True</code>	<code>False</code> （如果混入敏感工具）

8. 添加退出节点 `leave_skill`

9. 添加退出节点到主助理间的边

注：因为退出节点定义后所有子助力都可以使用，所以构建后续子工作流时，8、9两步不需要再次执行

航班助手的子工作流代码如下：

```
# 航班助手的 子工作流
def build_flight_graph(builder: StateGraph) -> StateGraph:
    """构建 航班预订助理的子工作流图"""
    # 添加入口节点，当需要更新或取消航班时使用
    builder.add_node(
        "enter_update_flight",
        create_entry_node("Flight Updates & Booking Assistant",
            "update_flight"), # 创建入口节点，指定助理名称和新对话状态
    )
    builder.add_node("update_flight",
        CtripAssistant(update_flight_runnable)) # 添加处理航班更新的实际节点
    builder.add_edge("enter_update_flight", "update_flight") # 连接入口节点到实际处理节点

    # 添加敏感工具和安全工具的节点
    builder.add_node(
        "update_flight_sensitive_tools",
        create_tool_node_with_fallback(update_flight_sensitive_tools),
    )
    # 敏感工具节点，包含可能修改数据的操作
```

```

    )
    builder.add_node(
        "update_flight_safe_tools",
        create_tool_node_with_fallback(update_flight_safe_tools), # 安全
工具节点，通常只读查询
    )

def route_update_flight(state: dict):
    """
    根据当前状态路由航班更新流程。

    :param state: 当前对话状态字典
    :return: 下一步应跳转到的节点名
    """

    route = tools_condition(state) # 判断下一步的方向
    if route == END:
        return END # 如果结束条件满足，则返回END
    tool_calls = state["messages"][-1].tool_calls # 获取最后一条消息中的
工具调用
    did_cancel = any(tc["name"] == CompleteOrEscalate.__name__ for
tc in tool_calls) # 检查是否调用了CompleteOrEscalate
    if did_cancel:
        return "leave_skill" # 如果用户请求取消或退出，则跳转至
leave_skill节点
    safe_tool_names = [t.name for t in update_flight_safe_tools] #
获取所有安全工具的名字
    if all(tc["name"] in safe_tool_names for tc in tool_calls): # 如
果所有调用的工具都是安全工具
        return "update_flight_safe_tools" # 跳转至安全工具处理节点
    return "update_flight_sensitive_tools" # 否则跳转至敏感工具处理节点

# 添加边，连接敏感工具和安全工具节点回到航班更新处理节点
builder.add_edge("update_flight_sensitive_tools", "update_flight")
builder.add_edge("update_flight_safe_tools", "update_flight")

# 根据条件路由航班更新流程
builder.add_conditional_edges(
    "update_flight",
    route_update_flight,
    ["update_flight_sensitive_tools", "update_flight_safe_tools",
"leave_skill", END],
)

# 此节点将用于所有子助理的退出
def pop_dialog_state(state: dict) -> dict:
    """
    弹出对话框并返回主助理。
    这使得完整的图可以明确跟踪对话流，并根据需要委托控制给特定的子图。

    :param state: 当前对话状态字典
    :return: 包含新的对话状态和消息的字典
    """

    messages = []
    if state["messages"][-1].tool_calls:
        # 注意：目前不处理LLM同时执行多个工具调用的情况
        messages.append(
            ToolMessage(

```

```

        content="正在恢复与主助理的对话。请回顾之前的对话并根据需要协助用户。",
        tool_call_id=state["messages"][-1].tool_calls[0]
["id"],
    )
    )
    return {
        "dialog_state": "pop", # 更新对话状态为弹出
        "messages": messages, # 返回消息列表
    }

# 添加退出技能节点，并连接回主助理
builder.add_node("leave_skill", pop_dialog_state)
builder.add_edge("leave_skill", "primary_assistant")
return builder

```

5定义主agent

5.1prompt

prompt逻辑和子agent一致

5.2Tools

定义一个工具包含搜索、政策查询、航班查询的基础工具，工具节点使用1.6中定义好的工具节点函数封装（保护性代码）

5.3构建主agent

与子agent不同的是，在绑定prompt和Tools后，不需要绑定 `CompleteOrEscalate`，而是绑定各个子agent的数据模型类，用于后续由主agent向各个子agent路由时判断

主agent代码如下

```

# 主助理提示模板
primary_assistant_prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "您是携程瑞士航空公司的客户服务助理。"
            "您的主要职责是搜索航班信息和公司政策以回答客户的查询。"
            "如果客户请求更新或取消航班、预订租车、预订酒店或获取旅行推荐，请通过调用相应的工具将任务委派给合适的专门助理。您自己无法进行这些类型的更改。"
            "只有专门助理才有权限为用户执行这些操作。"
            "用户并不知道有不同的专门助理存在，因此请不要提及他们；只需通过函数调用来安静地委派任务。"

            "向客户提供详细的信息，并且在确定信息不可用之前总是复查数据库。"
            "在搜索时，请坚持不懈。如果第一次搜索没有结果，请扩大查询范围。"
            "如果搜索无果，请扩大搜索范围后再放弃。"
            "\n\n当前用户的航班信息:\n<Flights>\n{user_info}\n</Flights>"
            "\n\n当前时间: {time}."
        ),
        ("placeholder", "{messages}"),
    ]
)

```

```

    ]
).partial(time=datetime.now())

# 定义主助理使用的工具
primary_assistant_tools = [
    tavily_tool, # 假设TavilySearchResults是一个有效的搜索工具
    search_flights, # 搜索航班的工具
    lookup_policy, # 查找公司政策的工具
]

# 创建可运行对象，绑定主助理提示模板和工具集，包括委派给专门的助理的工具
assistant_runnable = primary_assistant_prompt | llm.bind_tools(
    primary_assistant_tools
    + [
        ToFlightBookingAssistant, # 用于转交航班更新或取消的任务
        ToBookCarRental, # 用于转交租车预订的任务
        ToHotelBookingAssistant, # 用于转交酒店预订的任务
        ToBookExcursion, # 用于转交旅行推荐和其他游览预订的任务
    ]
)

```

bind_tools 作用：将Pydantic模型类转化为LLM可调用的工具，扩展主助理的能力边界

5.4数据模型类

数据模型类代码如下，仅展示航班与租车的

```

class ToFlightBookingAssistant(BaseModel):
    """
    将工作转交给专门处理航班查询，更新和取消的助理。
    """

    request: str = Field(
        description="更新航班助理在继续之前需要澄清的任何后续问题。"
    )

class ToBookCarRental(BaseModel):
    """
    将工作转交给专门处理租车预订的助理。
    """

    location: str = Field(
        description="用户想要租车的位置。"
    )
    start_date: str = Field(description="租车开始日期。")
    end_date: str = Field(description="租车结束日期。")
    request: str = Field(
        description="用户关于租车的任何额外信息或请求。"
    )

class Config:
    json_schema_extra = {
        "示例": {
            "location": "巴塞尔",

```



```
        "start_date": "2023-07-01",
        "end_date": "2023-07-05",
        "request": "我需要一辆带自动变速器的小型车。",
    }
}
```

6 定义最终的工作流

首先定义一个流程图的构建对象

```
# 定义了一个流程图的构建对象
builder = StateGraph(State)
```

6.1 定义初始信息获取节点

定义函数 `get_user_info` 调用自定义的用户基本航班信息的获取函数，并将函数的查询结果加入 `user_info` 中返回

将 `get_user_info` 作为节点添加进工作流中，并且添加 `START` 到该节点的边，每次对话开始前都先执行该节点，后续对话中所有agent均可在 `user_info` 获取用户基本航班信息，不用再次查询

代码如下

```
def get_user_info(state: State):
    """
    获取用户的航班信息并更新状态字典。
    参数：
        state (State): 当前状态字典。
    返回：
        dict: 包含用户信息的新状态字典。
    """
    return {"user_info": fetch_user_flight_information.invoke({})}

# 新增: fetch_user_info节点首先运行，这意味着我们的助手可以在不采取任何行动的情况下看到用户的航班信息
builder.add_node('fetch_user_info', get_user_info)
builder.add_edge(START, 'fetch_user_info')
```

6.2 在工作流中添加子工作流

调用定义好的构建子工作流的函数，将四个业务助理 的子工作流添加到工作流中

```
# 添加 四个业务助理 的 子工作流
builder = build_flight_graph(builder)
builder = builder_hotel_graph(builder)
builder = build_car_graph(builder)
builder = builder_excursion_graph(builder)
```

6.3添加主agent

1. 添加定义好的主agent节点
2. 添加主助理工具的节点
3. 添加主助力到四个子助理的条件边

定义路由函数，首先判断下一步是否为 `END`，然后判断最后一条工具调用是否是之前定义的几个子助理的工具模型类，都不是的话路由至主助理的工具节点

4. 添加主助理工具返回主助理的边

```
def route_primary_assistant(state: dict):
    """
    根据当前状态 判断路由到 子助手节点。
    :param state: 当前对话状态字典
    :return: 下一步应跳转到的节点名
    """

    route = tools_condition(state) # 判断下一步的方向
    if route == END:
        return END # 如果结束条件满足，则返回END
    tool_calls = state["messages"][-1].tool_calls # 获取最后一条消息中的工具调用
    if tool_calls:
        if tool_calls[0]["name"] == ToFlightBookingAssistant.__name__:
            return "enter_update_flight" # 跳转至航班预订入口节点
        elif tool_calls[0]["name"] == ToBookCarRental.__name__:
            return "enter_book_car_rental" # 跳转至租车预订入口节点
        elif tool_calls[0]["name"] == ToHotelBookingAssistant.__name__:
            return "enter_book_hotel" # 跳转至酒店预订入口节点
        elif tool_calls[0]["name"] == ToBookExcursion.__name__:
            return "enter_book_excursion" # 跳转至游览预订入口节点
        return "primary_assistant_tools" # 否则跳转至主助理工具节点
    raise ValueError("无效的路由") # 如果没有找到合适的工具调用，抛出异常

builder.add_conditional_edges(
    'primary_assistant',
    route_primary_assistant,
    [
        "enter_update_flight", # 航班 子助手的入口节点
        "enter_book_car_rental", # 租车 子助手的入口节点
        "enter_book_hotel", # 酒店 子助手的入口节点
        "enter_book_excursion", # 旅游景点 子助手的入口节点
        "primary_assistant_tools", # 主助手的工具： 全网搜索工具，查询企业政策的工
具
    ],
    END,
)

builder.add_edge('primary_assistant_tools', 'primary_assistant')
```

6.4定义从fetch_user_info节点到下一节点的边

用户提问后，工作流首先会执行 `fetch_user_info` 节点获取航班基本信息，下面就是要添加获取信息后应该去哪个节点

定义路由函数，获取当前对话状态中的 `dialog_state`

- 如果为空，表示目前还没有节点的调用，于是返回主助理节点，即接下来由主助理来对话
- 如果不为空，则返回最后一个对话状态，即 `dialog_state` 中的栈顶元素，即为接下来要调用的自助理节点

需求	实现方式	解决的问题
上下文保持	通过 <code>dialog_state</code> 栈跟踪活跃工作流	避免用户响应被错误路由
无缝回落	无状态时返回主助理	防止系统卡死在无效状态
动态扩展性	支持任意嵌套的子工作流	新增子助手无需修改路由逻辑

代码如下

```
# 每个委托的工作流可以直接响应用户。当用户响应时，我们希望返回到当前激活的工作流
def route_to_workflow(state: dict) -> str:
    """
    如果我们在一个委托的状态中，直接路由到相应的助理。
    :param state: 当前对话状态字典
    :return: 应跳转到的节点名
    """

    dialog_state = state.get("dialog_state")
    if not dialog_state:
        return "primary_assistant" # 如果没有对话状态，返回主助理
    return dialog_state[-1] # 返回最后一个对话状态

builder.add_conditional_edges("fetch_user_info", route_to_workflow) # 根据获取用户
信息进行路由
```

6.5保存对话并设置中断

创建一个内存检查点(checkpointer)用于保存对话状态

- `MemorySaver` 是一个轻量级的状态存储器
- 会以 `session_id` 为键保存每次对话的状态快照
- 支持对话中断后恢复

工作流编译

- `checkpointer=memory`: 指定使用刚创建的memory作为状态存储器
- 对于敏感工具，需要在调用前向用户确认，故添加中断。 `interrupt_before`: 在这些敏感工具执行前暂停工作流

```
memory = MemorySaver()
```

```

graph = builder.compile(
    checkpointer=memory,
    interrupt_before=[
        "update_flight_sensitive_tools",
        "book_car_rental_sensitive_tools",
        "book_hotel_sensitive_tools",
        "book_excursion_sensitive_tools",
    ]
)

#
# draw_graph(graph, 'graph4.png')

session_id = str(uuid.uuid4())
update_dates() # 每次测试的时候：保证数据库是全新的，保证，时间也是最近的时间

# 配置参数，包含乘客ID和线程ID
config = {
    "configurable": {
        # passenger_id用于我们的航班工具，以获取用户的航班信息
        "passenger_id": "3442 587242", #模拟用户访问
        # 检查点由session_id访问
        "thread_id": session_id,
    }
}

```

7执行工作流

1. 检查用户输入，如果为'q', 'exit', 'quit'则结束对话
2. **graph.stream()**：执行工作流，将用户输入注入系统，推动状态机前进，首次 `stream()` 执行完成全部工作（无敏感节点）
 - 输入：`{'messages': ('user', question)}` 将用户输入格式化为消息
 - `config`：包含用户ID和会话ID的配置
 - `stream_mode='values'`：以值形式流式返回事件
3. 敏感操作确认，通过 `current_state.next` 获取当前状态的下一个待执行节点
 - 如果值为None则表示工作流正常执行，无需中断，不会执行 `if current_state.next` 的后续代码。
 - 其余情况可以中断，询问用户是否批准操作
 - 用户批准后继续执行工作流，此时不需要重复传入用户输入，即参数input传None，然后打印事件消息
 - 用户拒绝执行敏感操作，则构造一个工具拒绝消息ToolMessage，并将拒绝原因反馈给工作流，然后继续执行工作流处理拒绝后的逻辑

代码如下

```

_printed = set() # set集合，避免重复打印

# 执行工作流
while True:
    question = input('用户: ')

```

```

if question.lower() in ['q', 'exit', 'quit']:
    print('对话结束，拜拜！')
    break
else:
    events = graph.stream({'messages': ('user', question)}, config,
stream_mode='values')
    # 打印消息
    for event in events:
        _print_event(event, _printed)

current_state = graph.get_state(config)
if current_state.next:
    user_input = input(
        "您是否批准上述操作？输入'y'继续；否则，请说明您请求的更改。\\n"
    )
    if user_input.strip().lower() == "y":
        # 继续执行
        events = graph.stream(None, config, stream_mode='values')
        # 打印消息
        for event in events:
            _print_event(event, _printed)
    else:
        # 通过提供关于请求的更改/改变主意的指示来满足工具调用
        result = graph.stream(
            {
                "messages": [
                    ToolMessage(
                        tool_call_id=event["messages"][-1].tool_calls[0]
["id"],
                        content=f"Tool的调用被用户拒绝。原因：'{user_input}'。",
                    )
                ],
            },
            config,
        )
        # 打印事件详情
        for event in result:
            _print_event(event, _printed)

```